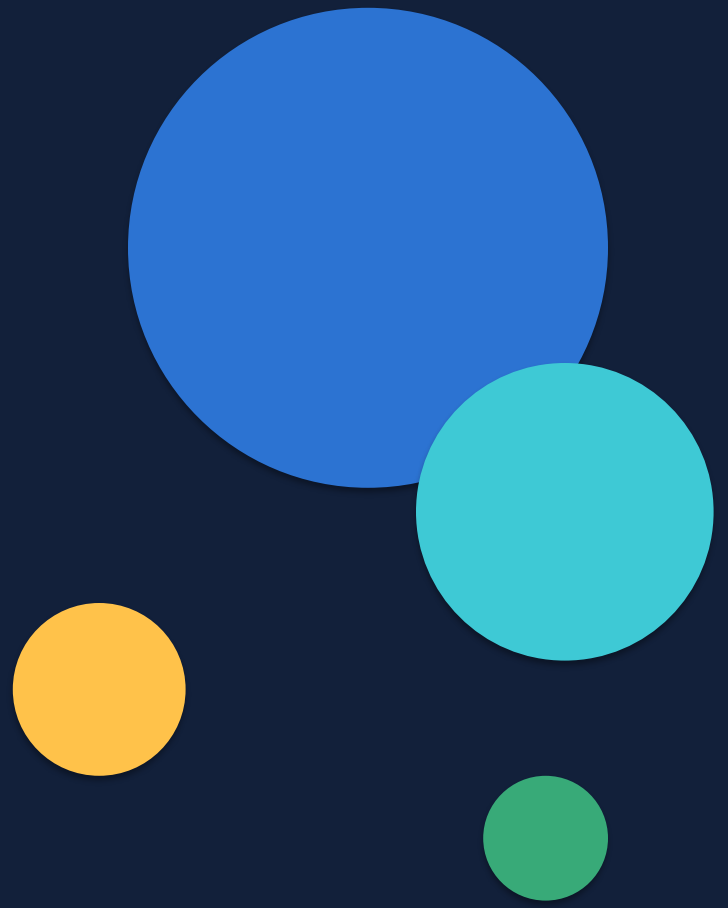
A BEGINNER-FRIENDLY GUIDE

INTRODUCTION TO PYTHON

From programming language basics to writing your first useful programs



A repeatable learning pattern: Definition → Explanation → Examples → Implementation

PRESENTATION

What is a programming language?

DEFINITION

A programming language is a formal way to write instructions that a computer can execute.

Low-level languages

Closer to hardware; give detailed control.

Examples: machine code, assembly.

High-level languages

More readable and convenient for people.

Examples: Python, Java, C#, JavaScript.

Language by purpose

Web: JavaScript
Systems: C, Rust
Data: Python, R
Databases: SQL

Practical analogy: a programming language is like a recipe format—clear instructions help produce a predictable result.

How code runs: compiled vs. interpreted

DEFINITION

These terms describe common ways source code is translated and executed.

Compiled approach

Explanation: a compiler translates code into another form before execution, often machine code.

Examples: C, C++, Rust. Java and C# commonly compile to intermediate code.

Interpreted approach

Explanation: code is executed through a runtime/interpreter.

Examples: Python, JavaScript, Ruby. CPython usually compiles to bytecode, then executes it in its virtual machine.

PRACTICAL TAKEAWAY

The distinction is useful but not absolute: language implementations may combine compilation and interpretation.

A brief history of Python

DEFINITION / CONTEXT

Python is a high-level, general-purpose language created by Guido van Rossum and first released publicly in 1991.

Late 1980s

Development begins at CWI in the Netherlands.

1991

First public release of Python.

2000

Python 2.0 adds major features and community-led development.

2008

Python 3.0 brings significant changes and is not fully backward-compatible with Python 2.

TODAY

Used in automation, web development, data science, and AI.

FUN FACT

The name came from Monty Python, the British comedy group—not the snake.

Why learn Python?

DEFINITION

Python is a programming language used to give computers instructions.

EXPLANATION

Python is popular because its code is readable and it can be used for many tasks.

BOOKISH EXAMPLE

Tell Python to display a greeting: print('Hello!')

PRACTICAL EXAMPLE

A program can greet a user when they open an app.

IMPLEMENTATION

print('Welcome!')

Output: Welcome!

Getting started: where to write Python

Definition

A development environment is where you write and run your code.

Explanation

You can install Python locally or use an online notebook to begin without setup.

Bookish example

A lesson might ask you to save a script as hello.py and run it.

Practical example

Use a browser notebook for a quick experiment, or install Python from python.org for local projects.

```
print('Hello, world!')
```

Implementation

Run it and confirm the text appears.

Your first program: print()

1 DEFINITION

print() is a built-in function that displays information.

2 EXPLANATION

Put text in quotes; Python executes the statement and shows the result.

3 BOOKISH EXAMPLE

Display a greeting
print('Hello!') displays Hello!

4 LIVE / PRACTICAL EXAMPLE

Welcome a new teammate
Display a personalized welcome message.

```
print('Welcome to Python!')
print('Let us start coding.')
```

5 IMPLEMENTATION / TRY IT

Change the message inside the quotes and run it again.

Variables and data types

1 DEFINITION

A variable is a name that refers to a value; a data type describes the kind of value.

2 EXPLANATION

Variables let a program store and reuse information. Common types include str, int, float, and bool.

3 BOOKISH EXAMPLE

Store a student's details
name = 'Ali' | age = 20

4 LIVE / PRACTICAL EXAMPLE

Track an order
Store its number, total price, and whether it has shipped.

```
name = 'Ali'          # str
items = 3             # int
price = 12.50         # float
shipped = False       # bool
```

5 IMPLEMENTATION / TRY IT

Variable naming conventions

A variable name should be clear, meaningful, and follow the language's naming rules.

snake_case

first_name • total_price
Words are separated by underscores.

camelCase

firstName • totalPrice
First word starts lowercase; later words are capitalized.

PascalCase

FirstName • TotalPrice
Each word starts with a capital letter.

kebab-case

first-name • total-price
Words are separated by hyphens; not valid for ordinary Python variable names.

Python conventions and rules

Use snake_case

Recommended for Python variables and functions.

Use PascalCase

Commonly used for Python class names.

Rules: names cannot start with a digit, cannot contain spaces or hyphens, and are case-sensitive. Avoid Python keywords such as class or for.

Operators

1 DEFINITION

Operators are symbols or keywords that perform calculations or comparisons.

2 EXPLANATION

Arithmetic operators calculate; comparison operators return True or False; logical operators combine conditions.

3 BOOKISH EXAMPLE

Calculate a total
10 + 3 gives 13; 10 > 3 gives True.

4 LIVE / PRACTICAL EXAMPLE

Calculate a shopping basket
Multiply price by quantity to find a line total.

```
price = 250
quantity = 3
total = price * quantity
print(total)  # 750
```

5 IMPLEMENTATION / TRY IT

Try +, -, *, /, ==, and > with different values.

Conditions: if / else

1 DEFINITION

Conditional statements run different code depending on whether a condition is true.

2 EXPLANATION

Python checks the condition; indentation marks which statements belong to each branch.

3 BOOKISH EXAMPLE

Check a passing score
If score is at least 50, display Pass; otherwise display Try again.

4 LIVE / PRACTICAL EXAMPLE

Approve a discount
Apply a discount only when the basket total reaches a threshold.

```
score = 75
if score ≥ 50:
    print('Pass')
else:
    print('Try again')
```

5 IMPLEMENTATION / TRY IT

Change score to 40 and observe how the output changes.

Loops

1 DEFINITION

A loop repeats a block of code.

2 EXPLANATION

A for loop commonly iterates over a sequence or range; a while loop repeats while a condition remains true.

3 BOOKISH EXAMPLE

Count from 1 to 3
A for loop can visit each number in range(1, 4).

4 LIVE / PRACTICAL EXAMPLE

Process several orders
Repeat the same calculation for every order in a list.

```
for number in range(1, 4):
    print(number)

# Output: 1, then 2, then 3
```

5 IMPLEMENTATION / TRY IT

Change the range to range(1, 6). Notice the stop value is excluded.

Lists and dictionaries

1 DEFINITION

A list stores an ordered collection; a dictionary stores key–value pairs.

2 EXPLANATION

Lists are useful for similar items; dictionaries label values with meaningful keys.

3 BOOKISH EXAMPLE

Store fruit names
fruits = ['apple', 'mango', 'banana']

4 LIVE / PRACTICAL EXAMPLE

Represent a customer record
A dictionary can keep a customer's name and city together.

```
fruits = ['apple', 'mango', 'banana']
print(fruits[0])  # apple

customer = {'name': 'Sara', 'city': 'Lahore'}
print(customer['city'])
```

5 IMPLEMENTATION / TRY IT

Try changing the first list item or reading another dictionary key.

Functions

1 DEFINITION

A function is a named, reusable block of code that can accept inputs and return a result.

2 EXPLANATION

Functions reduce repetition and help break a larger problem into smaller parts.

3 BOOKISH EXAMPLE

Convert Celsius to Fahrenheit
A function can take a temperature and return its conversion.

4 LIVE / PRACTICAL EXAMPLE

Reuse a shipping-cost calculation
Call the same function for multiple orders.

```
def greet(name):
    return f'Hello, {name}!'

print(greet('Sara'))
```

5 IMPLEMENTATION / TRY IT

Call greet() with a different name. Then try writing a function of your own.

Errors and debugging

1 DEFINITION

Debugging is the process of finding and fixing problems in a program.

2 EXPLANATION

Syntax errors break language rules; runtime errors occur during execution; logic errors produce incorrect results.

3 BOOKISH EXAMPLE

Find a syntax error
A missing colon after an if statement can stop the program.

4 LIVE / PRACTICAL EXAMPLE

Check a report total
If a total looks wrong, test inputs and inspect each calculation.

```
price = 250
quantity = 3
total = price * quantity
print('Total:', total)
```

5 IMPLEMENTATION / TRY IT

Change one value at a time and read error messages carefully.

Putting concepts together: order total

1 DEFINITION

A small program combines variables, arithmetic, and output to solve a task.

2 EXPLANATION

Store a unit price and quantity, multiply them, then display the result.

3 BOOKISH EXAMPLE

Price = 250; quantity = 3; total = 750.

4 PRACTICAL EXAMPLE

Calculate the cost of three notebooks priced at 250 each.

```
unit_price = 250
quantity = 3
total = unit_price * quantity
print('Order total:', total)
```

5 IMPLEMENTATION

Run the code, then change the price or quantity. Next, add a discount.

Your learning roadmap

Syntax & variables

Print, values, types, operators

Control flow

Conditions and loops

Data structures

Lists, dictionaries, sets, tuples

Functions & modules

Reusable code and imports

Files & errors

Read/write files, handle exceptions

Build a project

Calculator, quiz, or report automation

Consistency beats cramming write a little code every day.

KEY TAKEAWAYS

Start simple.
Stay curious.
Keep building.

```
>>> print('Hello, Python!')
Hello, Python!
```

Python becomes easier when you practice concepts by writing and changing small programs.

Questions?

THANK YOU